Python 十课复习手册 · 跟练版（双语 · 打印批注版）

张双 | 2026-10-28 起步 → 2026-11-01 | 阶段一~三完成 + 阶段四进行中 | 术语格式：英文（中文）

一、知识地图：10 节课一张图

L1 · 10-28

输出与输入 print · input

- `print()` 打印输出，`input()` 接收输入
- `#` 注释 comment；代码符号必须英文半角 half-width
- 改完代码要 `Cmd+S` 保存 save 再运行 run

坑：中文标点混入代码 → SyntaxError（语法错误）

L2 · 10-28

变量与类型 variable · type

- `str` 字符串 string / `int` 整数 integer / `float` 小数 floating-point
- `input()` 一律返回 str，算数前要 `int()` 或 `float()` 转换 convert
- print 里逗号自动加空格；`+` 拼接 concatenate 必须都是 str

坑：不转 int 就加减 → TypeError（类型错误）

L3 · 10-28

分支判断 if · elif · else

- `if / elif / else` 如果 / 否则如果 / 否则
- `==` 比较 compare，`=` 赋值 assign
- 缩进 indentation 4 空格是 Python 语法（不是装饰）

坑：忘缩进 → IndentationError（缩进错误）

L4 · 10-28

循环与随机 loop · random

- `while True:` 循环 + `break` 跳出
- `random.randint(1, 100)` 随机整数
- 计数器 counter：`count = count + 1`

坑：死循环 infinite loop → Ctrl+C 终止

L5 · 10-29

容错处理 try · except

- 可能出错的代码放进 `try:` 尝试
- `except:` 接住错误 + `continue` 继续下一轮
- 程序不再被坏输入打崩 crash

坑：except 后没 continue → NameError（名称错误）

L6 · 10-29

文件读写 file · open · write

- `open("日记.txt","a",encoding="utf-8")` 打开
- `"a"` 追加 append / `"w"` 覆盖 write / `"r"` 读取 read
- `readlines()` 逐行读，`strip()` 去掉首尾空白

坑：write() 只收 1 个参数 argument，不能像 print 用逗号

L7 · 10–30

循环统计 for · split · len

- `for line in lines:` 遍历 iterate 每一行
- `line.split()` 按空格切分 split 出名称和金额
- `len()` 长度 length、`round(x,1)` 四舍五入

坑："过期变量" stale variable；脏数据 dirty data → 循环内 try/except 跳过 skip

L8 · 10–30

字典 dict · dictionary

- `money_sum = {}` 建空字典、`name in dict` 查键 key 存在
- 核心套路：有就累加 accumulate，没有就新建 create
- 延伸：`max()` 最大 / `min()` 最小 找价差

坑：变量名拼错 typo → 看 Problems（问题）面板

L9 · 11–01

Git 本地 add · commit

- 三区：工作区 working → 暂存区 staging → 仓库 repository
- `git status` 状态 / `diff` 差异 / `log` 日志
- "没输出 = 成功"的 Unix 哲学

坑：配置后必验证 verify；`add .` 只管当前目录，全仓库用 `add -A`

L10 · 11–01

GitHub 远程 remote · push · pull

- `commit` 本地存档，`push` 推送上云，`pull` 拉取回来
- 日常三部曲：`add → commit → push`
- VS Code OAuth 授权 authorize 免配置密钥

坑：Public 公开 / Private 私有 可见性；不用的 PAT（个人令牌）要吊销

L11 · 11–01

虚拟环境 + pandas venv · DataFrame

- `source .venv/bin/activate` 激活 → `deactivate` 退出
- `read_csv / read_excel / groupby / to_excel`
- 向量化 vectorization：整列运算，不用 for 循环

坑：忘进 venv → ModuleNotFoundError（找不到模块）

阶段四 · 进行中

物资核算 + 光伏成本分析 project

- 查重 `drop_duplicates()` 去重 deduplicate（报清单，不静默删）
- 汇总 `groupby().agg()` 聚合、加权平均 weighted average
- `df.loc[条件, 列]` 按条件改值、openpyxl 标红 highlight

坑：垃圾进垃圾出 GIGO——数据质量是第一道防线

二、跟练方法（20 分钟一轮）

三轮法：① 闭卷在终端 terminal 里敲一遍（卡住也要自己写完）；② 点"查看参考答案"对照；③ 差异用 `git commit` 记下。
原则：不抄答案、先写再看；每课 1 道默写 dictation + 1 道变式 variation；错在哪就在那题旁边做个记号，下次只复习记号题。

三、跟练题库（点击展开答案）

阶段一 · 基础语法（L1–L5）

默写 **L1 · 输入 input 名字和项目名，输出 print 一句欢迎语** 3 分钟

预期运行效果：输入"张双"和"光伏项目"，输出：张双，欢迎回到光伏项目！

```
name = input("你叫什么？ ")
proj = input("什么项目？ ")
print(name, "，欢迎回到", proj, "！")
```

默写 **L2 · 输入两次金额，输出合计（保留 1 位小数）** 3 分钟

考：input 返回 str，必须先转换；金额可能带小数，用 float 浮点数

```
a = float(input("第一笔： "))
b = float(input("第二笔： "))
print("合计", round(a + b, 1), "元")
```

变式 **L2 · 同样输入两笔，多算一个"平均"** 3 分钟

```
avg = (a + b) / 2
print("平均", round(avg, 1), "元")
```

默写 **L3 · 输入分数，输出 A(≥90) / B(≥60) / C(其余)** 4 分钟

```
score = int(input("分数： "))
if score >= 90:
    print("A")
elif score >= 60:
    print("B")
else:
    print("C")
```

变式 L3 · 改成按"超耗率"分级：>5% 红、0~5% 黄、≤0 绿 4 分钟

```
rate = float(input("超耗率%: "))
if rate > 5:
    print("红（超耗）")
elif rate > 0:
    print("黄（关注）")
else:
    print("绿（正常）")
```

默写 L4 · 猜数字：随机 1~100，循环猜，提示大小，统计次数 6 分钟

```
import random
answer = random.randint(1, 100)
count = 0
while True:
    guess = int(input("猜一个数: "))
    count = count + 1
    if guess > answer:
        print("大了")
    elif guess < answer:
        print("小了")
    else:
        print("猜对了！用了", count, "次")
        break
```

变式 L4 · 最多猜 7 次，猜完没中就公布答案 4 分钟

提示：在循环开头加判断，break 前 print 答案

```
    if count >= 7:
        print("机会用完，答案是", answer)
        break
```

默写 L5 · 给猜数字加防崩（输入非数字不崩、重新输入） 4 分钟

```
    text = input("猜一个数: ")
    try:
        guess = int(text)
    except:
        print("请输入数字！")
        continue
```

阶段二 · 小脚本（L6–L8）

阶段二起点提醒（重要）：从 L6 开始，练习的性质变了——不再是"一问一答"，而是**读写文件的小程序**。所以每个练习都要（1）有配套的数据文件，（2）本身是完整可运行的程序。
① **先建数据文件** `流水.txt`（放在练习目录，每行"名称 金额"，其中故意留一条坏数据）：

```
奶茶 25
电影票 30
爆米花 abc
花生米 2
小苹果 6
香蕉 120
奶茶 10
```

② **片段不能单独跑**：像"跳过坏行"这种改造，必须放进 L7-01 的循环里；单独一个文件会报 `SyntaxError: 'continue' not properly in loop`。
③ **自检三问**：数据文件在不在？代码是否完整（有没有 `f.close()`）？这段代码跟哪个文件配套？

默写 L6 · 把输入的一句话追加 append 写入 note.txt 4 分钟

```
text = input("记点什么：  ")
f = open("note.txt", "a", encoding="utf-8")
f.write(text + "\n")
f.close()
print("写好了")
```

默写 L7 · 读流水文件（每行"名称 金额"），打印明细并算总额 6 分钟

```
f = open("流水.txt", "r", encoding="utf-8")
lines = f.readlines()
f.close()
total = 0
for line in lines:
    parts = line.split()
    name = parts[0]
    money = int(parts[1])
    print(name, money, "元")
    total = total + money
print("合计", total, "元")
```

变式 L7 · 查询时遇到坏行（金额不是数字）跳过 skip，不崩 4 分钟

```
    try:
        money = int(parts[1])
    except:
        print("（跳过坏数据：", line.strip(), "）")
        continue
```

默写 L8 · 用字典 dict 统计：奶茶 25, 电影票 30, 奶茶 10，按物品汇总 5 分钟

```
records = [("奶茶", 25), ("电影票", 30), ("奶茶", 10)]
money_sum = {}
for name, money in records:
    if name in money_sum:
        money_sum[name] = money_sum[name] + money
    else:
        money_sum[name] = money
print(money_sum)   # {'奶茶': 35, '电影票': 30}
```

变式 L8 · 同一物品出现不同价格就报警（奶茶：25 和 10） 4 分钟

```
prices = {}
for name, money in records:
    if name in prices:
        prices[name].append(money)
    else:
        prices[name] = [money]
for name in prices:
    if max(prices[name]) != min(prices[name]):
        print(name, "价格异常：", prices[name])
```

阶段三 · Git（L9—L10）

默写 L9 · 新项目第一次提交的完整命令（6 条） 4 分钟

```
git config --global user.name "张双"
git config --global user.email "zhangs0109@163.com"
git config --global core.quotepath false   # 中文文件名不乱码
git init
git add .
git commit -m "初始提交"
```

默写 L10 · 日常归档三部曲（今天改完代码后） 2 分钟

```
git add .
git commit -m "光伏项目成本分析 v1"
git push
```

变式 L9/L10 · 想撤销"暂存"和查看"改了什么"，各用什么命令？ 3 分钟

暂存区退回工作区 / 看本次改动内容

```
git status          # 先看状态
git diff            # 看具体改动（未暂存的）
git restore --staged 文件名    # 从暂存区撤回
```

L11 · 虚拟环境 venv + pandas

默写 L11 · 进环境 → 装库 → 读 CSV → 按列汇总 → 导出 Excel 6 分钟

```
# 终端里：
source .venv/bin/activate
pip install pandas openpyxl

# 代码里：
import pandas as pd
df = pd.read_csv("物品汇总.csv", encoding="utf-8-sig")
result = df.groupby("物品")["金额"].sum()
result.to_excel("汇总.xlsx")
```

变式 L11 · 报错 ModuleNotFoundError，第一反应查什么？ 1 分钟

不是代码问题，是环境 environment 问题

```
# 看终端提示符有没有（.venv）前缀；没有就：
source .venv/bin/activate
# 再跑
```

阶段四 · 项目实战（跟练改造）

改造 把 material_check.py 的超耗阈值 threshold 从 3% 调成 8% 5 分钟

只改文件顶部的配置项，不改逻辑

```
THRESHOLD = 8      # 原来 3
```

改造 给 pv_analysis.py 加一个维度：按"费用归集人员"统计 10 分钟

复用 agg_by 函数，一行就能加

```
staff = agg_by(["费用归集人员"])
# 再把它写进 Excel：staff.to_excel(w, sheet_name="11_归集人员", index=False)
```

应用 在 pv_analysis.py 里找 3 处可以自己改的地方，改完重跑对拍 15 分钟

例如：TOP20 改 TOP10、材料表 head(60) 改 head(100)、看板 KPI 加一张卡片、新增一张 Sheet

四、高频英文词汇表

Python 基础词

英文	中文	你的场景
variable	变量	`name = "张双"` 里的 name
string (str)	字符串	带引号的文本 `"奶茶"`
integer (int)	整数	次数、年份
float	浮点数（小数）	金额、单价、汇率
list	列表	`[25, 30, 10]` 一串数据
dict / dictionary	字典	`{"奶茶": 35}` 键值对 key–value
loop	循环	for / while
condition	条件	if 后面的判断
indentation	缩进	行首 4 个空格，Python 的语法
import	导入	`import pandas` 引入工具库
function	函数	`print()` 就是一个函数
argument / parameter	参数	括号里传给函数的东西
return	返回	函数算完交回来的结果
index	索引	从 0 开始的位置编号
row / column	行 / 列	DataFrame 的行列
missing / null	缺失 / 空	NaN 空值
duplicate	重复	重复录入的行
aggregate (agg)	聚合	groupby 后的汇总计算

Git 高频词

英文	中文	场景
repository (repo)	仓库	你的 python–learning 项目
working directory	工作区	你在改的文件
staging area	暂存区	git add 之后的中转站
commit	提交（存档）	git commit –m "说明"
push / pull	推送 / 拉取	上云 / 下载
remote	远程	origin 就是远程仓库的别名
branch	分支	main 是主分支
merge / conflict	合并 / 冲突	两人改同一处会冲突
tracked / untracked	已跟踪 / 未跟踪	文件后的 U 就是 untracked
modified	已修改	文件后的 M

常见报错 Error 对照（排错快一倍）

报错	中文含义	你遇到过的那次
SyntaxError	语法错误	中文标点混进代码
IndentationError	缩进错误	if 下面忘缩进 4 空格
NameError	名称错误（变量没定义）	except 后忘 continue；mongey_sum 拼错
TypeError	类型错误	write() 给了两个参数
ValueError	值错误	int("abc") 转不了数字
FileNotFoundError	找不到文件	cd 错目录、文件名写错
ModuleNotFoundError	找不到模块	忘进虚拟环境 (.venv)
KeyError	字典里没这个键	dict["不存在的名字"]
IndexError	索引越界	列表只有 3 个却取第 4 个

学习方法：不用背。每次报错时翻这页对照一次，三次就记住了。

五、扩展方向（按你的职业定制）

方向	做什么	难度
L12 · 函数 def (function)	把"读取→清洗→汇总→导出"封装成函数，下月换数据源只传文件名	★★
L13 · 月度环比 (month-over-month)	在光伏分析上加"本月 vs 上月"环比，用 `pct_change()`	★★
L14 · 分包结算台账	计量-应付-实付-余额四栏自动核算（复用 groupby + loc）	★★★
L15 · 供应商考核	把超耗追到供应商头上：同一供应商多笔合并、算单价离散度	★★★
L16 · 多文件合并 (merge)	十几个分包商各报一份 Excel，一个循环读进来 `pd.concat()`	★★★
通用 · 编码归一	把计价单位、材料编码建一张"标准字典"对照表，自动替换	★★

六、记忆曲线复习计划

时间	做什么	时长
今天	把这份手册按"三轮法"完整过一遍（本页）	20 分钟
3 天后（11–04）	只做标注过的错题 + 默写 L8/L11（字典、groupby 是核心）	10 分钟
1 周后（11–08）	不看答案重写物资核算 v2 的核心 10 行（查重+汇总+预警）	15 分钟
1 月后（12–01）	给光伏分析换一个真实新数据源，全流程独立复现	30 分钟

阿布的话：你已经从零基础走到"能自己改代码、能读懂 9000 行台账、能给工具挑毛病"——这比记住多少函数重要得多。接下来一个月，**不追新语法，只把这套东西用熟**。英文词汇不用背，遇到时回这页对照一次就够。